

S.O.001 · SESIUNEA 6 · ANUL I, SEM. 1

Informații invizibile: forensics și steganografie

Tehnici de programare · Ethical Hacking prin Command Line

Marius Dumitrașcu · marius@cybercor.org · Telegram +40 753 210 000



Ai capturat ceva. Dar nu se citește: e codat, amestecat, ascuns.

Săptămâna trecută am scos date dintr-o aplicație web: parole, cookie-uri, fișiere. Rar sunt în clar. De cele mai multe ori sunt un strat de base64, un XOR, un hash, sau un mesaj pitit într-o poză. Azi învățăm să desfacem straturile, programatic, în Python.



Trei lucruri pe care lumea le confundă: codare, criptare, hash.

Codarea schimbă formă, oricine o desface (base64, hex). Criptarea cere o cheie ca să revii. Hash-ul merge într-o singură direcție: nu îl „decriptezi", îl ghicești. Dacă pleci cu distincția asta clară, jumătate din munca de forensics e făcută.



Pe disc, orice fișier e un șir de octeți.

O poză, un PDF, o parolă, un pachet de rețea: la bază, același lucru, un șir de octeți (numere de la 0 la 255). Cine știe să citească octeți citește orice fișier, indiferent de extensie. Unealtă de azi nu se uită la pictograma fișierului, se uită la octeții lui.

PAS CU PAS · SCARA DE PROGRAMARE

Unde suntem, unde urcăm

- Variabile, tipuri, control (if, for) ✓
- **Octeți și biți: date binare și operatori** ◀ AZI
- Colecții: list, dict, set
- Funcții și excepții
- Iteratori și generatoare
- Obiecte și clase (OOP)
- Alte limbaje: PowerShell, C, C#

CONCEPTUL ZILEI

Octeți și biți: date binare

CE ESTE

Textul pentru oameni este str (litere). Ce stă, de fapt, pe disc sau pe fir este bytes: un șir de numere, fiecare între 0 și 255, pe care le citești și le prelucrezi pe biți.

MODELUL MENTAL

str = o foaie scrisă, pentru ochi. bytes = cutia cu numere din spate. Treci dintr-una în altă cu .encode() și .decode().

OPERAȚII ȘI METODE

- b[0] dă un număr (codul octetului), nu o literă
- .encode() / .decode(): între str și bytes
- bytes.fromhex() / .hex(): octeți din/în hex
- b ^ cheie, b & 1: operatori pe octeți
- open(cale, "rb"): citești fișierul ca octeți

CÂND ÎL FOLOSEȘTI

Oricând atingi un fișier, o imagine, o parolă sau trafic. Regulă: citești cu „rb" și treci la text doar când ești sigur că octeții chiar sunt text.

✋ ÎNTREBARE PENTRU SALĂ

„base64" pe care îl vezi peste tot: e o formă de criptare?

Nu. E doar o altă haină pentru aceiași octeți. Oricine o dă jos, fără nicio cheie. Azi vedem exact de ce.

abc PYTHON · FUNDAMENTUL ZILEI

Două tipuri: text și octeți

```
>_ python

s = "Salut"          # str: text, litere
b = b"Salut"         # bytes: octeți bruți

s.encode()           # str   -> bytes
b.decode()           # bytes -> str

b[0]                 # 83  (un octet = un număr)
bytes([83, 97])      # b"Să"
```

CE FACE

str e text pentru oameni; bytes e ce stă, de fapt, pe disc sau pe fir.

.encode() și .decode() trec dintr-unul în altul.

Indexezi bytes și primești un număr (0-255), nu un caracter.

🎯 LA CE FOLOSEȘTE

Aproape orice eroare de începător în forensics vine din amestecarea celor două. Regulă: citești fișiere cu "rb" (bytes), treci la text doar când ești sigur că e text.

VOCABULARUL ZILEI

Trei operații care se confundă

OPERAȚIA	CE FACE	O DESFACI?
codare (base64, hex)	altă formă, aceiași octeți	dă, oricine, fără cheie
criptare (AES, XOR)	ascunde cu o cheie	doar cu cheia
hash (md5, sha256)	amprentă, o direcție	nu: doar ghicești

Prima întrebare în fața unui șir necunoscut: e doar recodat, e criptat, sau e un hash? Răspunsul schimbă unealtă.

COLȚUL DE C · SUB CAPOTĂ

Un octet e un număr: în C se vede gol

>_ C

```c
unsigned char b = 'S';
printf("%d\n", b);   // 83
printf("%c\n", b);   // S
printf("%02x\n", b); // 53 (hex)
```

CE E NOU ÎN C

În C, un caracter ESTE un număr (tipul char = un octet).

%d îl arată ca număr, %c ca literă, %x ca hexazecimal.

Aceeași valoare, trei haine: exact ce facem azi cu fișierele.

ACELAȘI LUCRU ÎN PYTHON

În Python, b[0] îți dă direct numărul 83; în C, caracterul și numărul sunt deja același lucru.

De reținut: Un octet nu „e" literă sau număr; e o valoare 0-255 pe care noi o citim cum vrem.



PARTEA I

Straturi

Encoding: base64, hex, URL. Recunoști stratul, apoi îl desfaci în Python.

Șirul arată a „zgomot", dar are o formă.

Formă îți spune ce haină să dai jos.

abc PYTHON · BASE64

Stratul pe care îl vezi cel mai des

```
>_ python

import base64
s = "U2FsdXQgbHVtZQ=="
b = base64.b64decode(s)
print(b)            # b"Salut lume"
print(b.decode())   # Salut lume
```

CE FACE

base64 reprezintă octeți folosind 64 de caractere citibile.

b64decode întoarce bytes; .decode() îi face text (dacă e text).

Semnul = la coadă e umplutură, un indiciu puternic de base64.

🎯 LA CE FOLOSEȘTE

base64 apare peste tot: în cookie-uri, în token-uri, în fișiere de configurare. Nu ascunde nimic, doar împachetează octeți ca text. Dai jos stratul într-o linie.

🔎 **PYTHON · RECUNOAȘTERE**

Ce strat e? Îl citești după formă

```
>_ python

s = "4861 7368"          # ? hex
t = "SGVsbG8="           # ? base64
u = "%2Fetc%2Fpasswd"    # ? URL

# hex:    doar 0-9 a-f, lungime para
# base64: A-Za-z0-9+/ , poate termina cu =
# URL:    are %xx (ex. %2F = / )
```

CE FACE

Alfabetul trădează stratul: ce caractere apar și care lipsesc.

hex are doar 16 simboluri; base64 are 64 plus „="; URL are „%".

Nu ghici orbește: uită-te întâi la mulțimea de caractere.

🎯 **LA CE FOLOSEȘTE**

Un analist nu încearcă toate decodările la nimereală. Se uită la formă (alfabet, lungime, umplutură) și știe ce să încerce întâi. E prima abilitate de forensics.

🧩 PYTHON · HEX

Octeți scriși ca hexazecimal

>_ python

```
h = "48656c6c6f"
b = bytes.fromhex(h)
print(b)            # b"Hello"

# și invers:
print(b.hex())      # 48656c6c6f
```

CE FACE

hex scrie fiecare octet cu două caractere (00 până la ff).

bytes.fromhex citește șirul hex și întoarce octeții.

.hex() face drumul invers: octeți -> șir hex.

🎯 LA CE FOLOSEȘTE

Hexul e limba în care se arată octeții bruți: în editoare hex, în dump-uri de memorie, în semnăturile de fișier pe care le vedem în partea a patra.

🔄 PYTHON · URL-ENCODING

De ce apare %2F în adrese

```
>_ python

from urllib.parse import unquote, quote
u = "%2Fetc%2Fpasswd"
print(unquote(u))   # /etc/passwd

print(quote("/etc/passwd"))
#  %2Fetc%2Fpasswd
```

CE FACE

URL-encoding scrie caractere speciale ca %xx (xx = codul hex).

unquote le readuce la normal; quote le codează.

%2F e „/", %20 e spațiu, %00 e octetul nul.

🎯 LA CE FOLOSEȘTE

Același truc apare și în atac: codezi un payload ca să treacă de un filtru. În S05 ai văzut „../" într-un parametru; adesea sosește codat ca „..%2F".

GHICEȘTE · VOTEAZĂ DIN SALĂ

Ce tipărește?

```
>_ python

import base64
s = "SGk="
print(base64.b64decode(s))
```

A b'Hi'

B 'Hi'

C SGk

D Aruncă eroare

✋ **Votează cu degetele: 1=A 2=B 3=C 4=D**

RĂSPUNSUL

Ce tipărește?

```
>_ python

import base64
s = "SGk="
print(base64.b64decode(s))
```

A `b'Hi'` ✓

B `'Hi'`

C `SGk`

D `Aruncă eroare`

De ce: b64decode întoarce întotdeauna BYTES, nu text: de aceea vezi prefixul b. Ca să obții "Hi" ca text, mai adaugi .decode(). Confuzia bytes/str, exact de la începutul orei.

🧅 PYTHON · STRATURI

Când decodarea dă tot zgomot: mai e un strat

```
>_ python

import base64
s = "NjE3MzYzNzU2ZTcz"
pas1 = base64.b64decode(s)
print(pas1)   # b"617363756e73" <- doar 0-9 a-f = hex
pas2 = bytes.fromhex(pas1.decode())
print(pas2)   # b"ascuns"  <- text! te opresti
```

CE FACE

Rezultatul unei decodări poate fi tot o codare.

Te oprești când iese text citibil (sau octeți de fișier cunoscut).

base64 -> hex -> text: aici stratul de sub hex era chiar mesajul.

🎯 **LA CE FOLOSEȘTE**

Provocările de forensics ascund mesajul sub mai multe straturi, exact ca o ceapă. Scrii o buclă care decodează până dă ceva citibil: regulă de oprire e „arată a text".

UNELTE REALE • LE VEI ÎNTÂLNI

Ce folosesc profesioniștii

Noi le scriem în Python ca să înțelegem mecanismul; în practică folosești și unelte gata făcute.



CyberChef

desface straturi de encoding în browser, vizual



xxd / hexdump

arată orice fișier ca octeți și hex



basenc / base64

codare/decodare din linia de comandă



file

ghicește tipul după conținut, nu după nume



binwalk

găsește fișiere ascunse în alt fișier



base64 / binascii

aceleași operații, în Python



PARTEA II

Cheia

XOR: operația care ascunde și dezvăluie cu aceeași cheie.
Spargerea prin forță brută.

base64 nu are cheie. XOR are.

Și, de multe ori, cheia e un singur octet.

🔑 PYTHON · XOR

Operația care se anulează pe ea însăși

```
>_ python

a = 0x41          # litera A
k = 0x10          # cheia
c = a ^ k         # 0x51 = Q (criptat)
print(c ^ k)      # 0x41 = A  (la loc!)

# proprietatea magica:  x ^ k ^ k == x
```

CE FACE

^ este XOR: compară bit cu bit, dă 1 unde biții diferă.

Același XOR cu aceeași cheie te duce și înapoi: criptezi și decriptezi la fel.

De aceea cine are cheia desface, iar operația e simetrică.

🎯 LA CE FOLOSEȘTE

XOR e peste tot în malware și în challenge-uri tocmai pentru că e simplu și reversibil. Cheia e secretul; fără ea, pare zgomot. Dar dacă e scurtă, nu rezistă.

GHICEȘTE · VOTEAZĂ DIN SALĂ

Cât face 0x41 ^ 0x20 ?

```
>_ python

a = 0x41    # A
print(chr(a ^ 0x20))
```

A 'a' (litera mică)

B 'A' (neschimbat)

C ' ' (spațiu)

D Eroare

✋ **Votează cu degetele: 1=A 2=B 3=C 4=D**

RĂSPUNSUL

Cât face 0x41 ^ 0x20 ?

```
>_ python

a = 0x41     # A
print(chr(a ^ 0x20))
```

A 'a' (litera mică) ✓

B 'A' (neschimbat)

C ' ' (spațiu)

D Eroare

De ce: Bitul 0x20 e exact diferența dintre majuscule și minuscule în ASCII: A=0x41, a=0x61. XOR cu 0x20 schimbă litera mare în mică și invers. Un truc clasic de citit octeți.

🔑 PYTHON · XOR PE OCTEȚI

Decriptezi un întreg mesaj cu o cheie

```
>_ python

date = bytes.fromhex("3223302d2e23")
cheie = 0x42
clar = bytes(b ^ cheie for b in date)
print(clar)          # b"parola"  cheie bună
print(clar.decode()) # parola
```

CE FACE

Parcurgi fiecare octet și îl treci prin XOR cu cheia.

bytes(...) adună rezultatele într-un nou șir de octeți.

errors="replace" nu crapă dacă un octet nu e text valid.

🎯 LA CE FOLOSEȘTE

Dacă rezultatul arată a text (litere, spații, semne de punctuație), ai nimerit cheia. Dacă e tot zgomot, nu e cheia potrivită. Asta ne duce direct la forța brută.

PYTHON · FORȚA BRUTĂ

Cheie de un octet: doar 256 variante

```
>_ python

date = bytes.fromhex("1e0b16165f...")
for k in range(256):
    clar = bytes(b ^ k for b in date)
    # păstrăm doar dacă pare text citibil:
    if all(32 <= c < 127 for c in clar):
        print(k, clar)
```

CE FACE

O cheie de un octet are fix 256 de valori posibile (0-255).

Le încerci pe toate și te uiți care dă text citibil.

32-126 e intervalul ASCII imprimabil: un filtru simplu.

🎯 LA CE FOLOSEȘTE

Nu „spargi" nimic complicat: lași calculatorul să încerce tot și alegi rezultatul care arată a limbă. Același tipar (încearcă tot, filtrează ce e plauzibil) se repetă în tot ce facem azi.

COLȚUL DE C · SUB CAPOTĂ

XOR în C: aceeași buclă, octet cu octet

```c
>_ C

unsigned char key = 0x42;
for (int i = 0; i < n; i++)
    buf[i] ^= key;   // in acelasi loc
// buf acum e clar (sau criptat)
```

CE E NOU ÎN C

buf[i] ^= key aplică XOR și scrie înapoi în același octet.

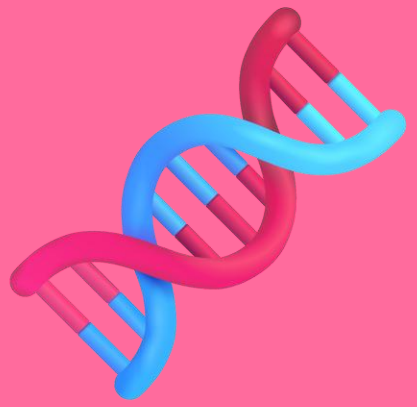
E aceeași buclă ca în Python, dar pe memoria fișierului direct.

Malware-ul chiar așa își ascunde șirurile: un XOR scurt la pornire.

ACELAȘI LUCRU ÎN PYTHON

bytes(b ^ k for b în date) din Python e exact această buclă, strânsă într-o linie.

De reținut: XOR „în loc" (^=) e preferatul malware-ului: puțin cod, ascunde șirurile de un antivirus naiv.



PARTEA III

Amprente

Hashing: ce e, cum îl recunoști, cum se „sparge" (nu se decriptează, se ghicește).

Unele lucruri nu se desfac niciodată.

Un hash merge într-o singură direcție.

PYTHON · HASHLIB

Amprenta unui mesaj

```
>_ python

import hashlib
h = hashlib.sha256(b"parola").hexdigest()
print(h)
#  a80b568a237f50391d2f1f97beaf99564e33d2e1...

# schimbi un singur octet -> amprenta total altă
```

CE FACE

sha256 ia octeți și dă o amprentă de lungime fixă.

hexdigest() o arată ca șir hex (64 de caractere la sha256).

Aceeași intrare dă mereu aceeași amprentă; o mică schimbare o schimbă complet.

🎯 LA CE FOLOSEȘTE

Hash-ul nu păstrează mesajul, ci o „amprentă" a lui. De aceea serverele nu stochează parola, ci hash-ul ei: la login, compară amprentele. Nu pot „citi" parola înapoi, dar nici tu, direct.

🔎 PYTHON · IDENTIFICARE

Ce hash e? Îl citești după lungime

```
>_ python

md5    = 32 caractere hex
sha1   = 40 caractere hex
sha256 = 64 caractere hex

h = "5f4dcc3b5aa765d61d8327deb882cf99"
print(len(h))   # 32  -> probabil md5
```

CE FACE

Fiecare algoritm dă o amprentă de lungime fixă.

Numeri caracterele hex și restrângi ce algoritm e.

32 -> md5, 40 -> sha1, 64 -> sha256 (dacă e hex).

🎯 LA CE FOLOSEȘTE

Primul pas în fața unui hash necunoscut: îi numeri lungimea. Îți spune ce algoritm să presupui și, deci, ce unealtă de spargere să pornești.

GHICEȘTE · VOTEAZĂ DIN SALĂ

Se poate „decripta" un hash sha256?

```
>_ python

h = hashlib.sha256(b"x")
      .hexdigest()
# pot află "x" din h?
```

A Nu direct: doar încercând intrări

B Da, cu funcția inversă

C Da, cu cheia

D Doar dacă e md5

✋ **Votează cu degetele: 1=A 2=B 3=C 4=D**

RĂSPUNSUL

Se poate „decripta" un hash sha256?

```
>_ python

h = hashlib.sha256(b"x")
      .hexdigest()
# pot află "x" din h?
```

A **Nu direct: doar încercând intrări ✓**

B Da, cu funcția inversă

C Da, cu cheia

D Doar dacă e md5

De ce: Un hash nu are inversă și nu are cheie: informația se pierde. Singurul drum înapoi e să încerci intrări (dicționar, forță brută) până dai de una cu aceeași amprentă. De asta parolele slabe cad repede.

PYTHON · SPARGERE

Cum „sparge" cineva un hash de parolă

```
>_ python

tinta = "5f4dcc3b5aa765d61d8327deb882cf99"
for cuv in open("rockyou.txt", encoding="latin1"):
    cuv = cuv.strip()
    if hashlib.md5(cuv.encode()).hexdigest() == tinta:
        print("gasit:", cuv)   # password
        break
```

CE FACE

Nu inversezi hash-ul: hashuiești fiecare cuvânt dintr-o listă și compari.

Când o amprentă coincide cu ținta, ai găsit parola.

rockyou.txt e listă clasică de parole scurse, folosită la testare.

🎯 LA CE FOLOSEȘTE

Puterea atacului stă în listă, nu în algoritm. O parolă din top-1000 cade în milisecunde. De aici regulă: parole lungi, unice, nu din dicționar. Apărarea o vedem imediat.

🧂 PYTHON · APĂRAREA

De ce „sarea" strică atacul

```
>_ python

import os, hashlib
sare = os.urandom(16)   # aleatorie, per user
h = hashlib.sha256(sare + b"parola").hexdigest()

# doi useri cu aceeași parola -> hash DIFERIT
# listele pre-calculate devin inutile
```

CE FACE

Sarea e un șir aleatoriu, adăugat la parolă înainte de hash.

Se păstrează lângă hash; nu e secretă, dar e unică per utilizator.

Rezultatul: aceeași parolă dă amprente diferite la doi oameni.

🎯 LA CE FOLOSEȘTE

Fără sare, atacatorul sparge o dată și prinde toți userii cu aceeași parolă. Cu sare, trebuie să atace fiecare cont separat, iar tabelele gata calculate (rainbow tables) nu mai ajută.

INTEGRITATE · NU DOAR PAROLE

HMAC: amprentă cu cheie



HMAC =

sigiliu cu cheie.

```
>_ integritate

import hmac, hashlib
cheie = b"secret-comun"
mesaj = b"transfera 100 lei"
t = hmac.new(cheie, mesaj,
             hashlib.sha256).hexdigest()
# trimiti mesaj + t; cine schimba mesajul,
# nu poate reface t fără cheie
```

Un hash simplu spune „mesajul nu s-a stricat pe drum". HMAC adaugă o cheie: spune „mesajul vine de la cine știe cheia, și nu a fost modificat".

UNELTE REALE · SPARGERE DE HASH

Ce vei folosi la laborator și pe platforme

Pe pwn.college și HTB vei dă de hash-uri des; unealtă potrivită pleacă de la algoritmul identificat.



hashid / hash-identifier

ghicește algoritmul după formă



john (John the Ripper)

spargere pe CPU, multe moduri



hashcat

spargere pe GPU, foarte rapid



CrackStation

caută hash-ul în baze deja sparte



bcrypt / argon2

cum SE STOCHEAZĂ corect parolele



hashlib / hmac

aceleași operații, în Python



PARTEA IV

Urme

Magic bytes, strings, file carving, metadate EXIF/GPS: ce spune un fișier despre el.

Un fișier nu e ce zice extensia.

E ce zic primii lui octeți.

PYTHON · MAGIC BYTES

Primii octeți spun tipul adevărat

```
>_ python

with open("misterios.dat", "rb") as f:
    cap = f.read(4)
print(cap)            # b"%PDF"  -> e un PDF!

# chiar dacă se numeste .dat sau .jpg
# semnatura din primii octeți nu minte
```

CE FACE

Fiecare format începe cu o semnătură fixă (magic bytes).

Citești primii câțiva octeți cu "rb" și o recunoști.

Extensia e doar o etichetă; semnătura e conținutul.

🎯 LA CE FOLOSEȘTE

Prima mișcare de forensics în fața unui fișier necunoscut: îi citești capul. O poză redenumită în .txt, un executabil deghizat în .jpg: toate se dau de gol prin primii octeți.

CATALOG · SEMNĂTURI

Magic bytes de știut pe de rost

FORMAT	PRIMII OCTEȚI (HEX)	CA TEXT
PNG	`89 50 4E 47`	.PNG
JPEG	`FF D8 FF`	(nu e text)
PDF	`25 50 44 46`	%PDF
ZIP / docx / jar	`50 4B 03 04`	PK..
ELF (binar Linux)	`7F 45 4C 46`	.ELF
GIF	`47 49 46 38`	GIF8

Un ZIP și un .docx au aceeași semnătură „PK": un docx chiar E o arhivă ZIP. Deschide-l ca ZIP și vezi.

🔎 PYTHON · STRINGS

Scoți textul citibil dintr-un binar

```
>_ python

date = open("program.bin", "rb").read()
import re
# șiruri de 4+ caractere imprimabile:
for m in re.findall(rb"[ -~]{4,}", date):
    print(m.decode())
#  http://c2.example.com  /tmp/.hidden ...
```

CE FACE

„strings" caută secvențe de caractere imprimabile într-un fișier binar.

rb"[-~]{4,}" e un regex pe BYTES: 4+ octeți din spațiu până la ~.

Scoate URL-uri, căi, mesaje, chei lăsate în clar.

🎯 LA CE FOLOSEȘTE

Prima unealtă pe orice binar necunoscut. Un malware îți scapă adesea adresă serverului de comandă, o cale, un mesaj, direct în strings. E ieftin și dă des rezultate.

COLȚUL DE C · SUB CAPOTĂ

De ce „rb": text vs binar, văzut din C

>_ C

```c
FILE *f = fopen("poza.png", "rb");
unsigned char cap[4];
fread(cap, 1, 4, f);
// cap[0]=0x89 cap[1]='P' ...
```

De reținut: Pentru fișiere care nu sunt text (poze, binare), „rb" nu e opțional: fără el, octeții se pot altera.

CE E NOU ÎN C

„rb" = read binary: iei octeții exact cum sunt, fără conversii.

Fără "b", unele sisteme „traduc" sfârșitul de linie și strică octeții.

fread umple un buffer cu octeți bruți, ca f.read() în Python.

ACELAȘI LUCRU ÎN PYTHON

open(path, "rb").read() din Python e exact acest fopen + fread: octeți neatinși, gata de inspectat.

✋ ÎNTREBARE PENTRU SALĂ

O poză pe care o postezi online: ce mai spune ea despre tine, în tăcere?

Adesea: modelul telefonului, dată exactă, și coordonatele GPS unde ai făcut-o. Totul scris în metadate.

📷 PYTHON · EXIF

Ce ascunde o fotografie în metadate

```
>_ python

from PIL import Image
from PIL.ExifTags import TAGS
img = Image.open("poza.jpg")
exif = img._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
#  Model = iPhone 13 ; DateTime = 2026:...
```

CE FACE

EXIF e un bloc de metadate pus de aparat în poză.

Pillow (PIL) îl citește; TAGS traduce numerele în nume citibile.

Conține model, dată, setări, și adesea coordonate GPS.

🎯 LA CE FOLOSEȘTE

OSINT-ul trăiește din asta: o singură poză poate trăda unde locuiește cineva. Invers, apărarea e simplă: ștergi metadatele înainte să publici. Îți arăt și GPS-ul.

📍 PYTHON · GPS

Din poză pe hartă

```
>_ python

gps = exif.get(34853, {})   # blocul GPS
# conține latitudine, longitudine ca
# (grade, minute, secunde) + emisfera
print(gps)
#  {1: "N", 2: (47,1,22), 3: "E", 4: (28,...)}
# -> 47.02.. N, 28.. E  (pui pe o harta)
```

CE FACE

Blocul GPS din EXIF ține latitudinea și longitudinea.

Vin ca grade, minute, secunde, plus emisfera (N/S, E/V).

Le convertești în grade zecimale și le pui pe o hartă.

🎯 LA CE FOLOSEȘTE

Un detector de GPS în poze e o unealtă de câteva linii, dar cu efect real: arată cât de mult publică oamenii fără să știe. De asta rețelele șterg EXIF la încărcare, iar tu ar trebui s-o faci oricum.

🧩 PYTHON · CARVING

Scoți un fișier ascuns în alt fișier

```
>_ python

date = open("suspect.png", "rb").read()
# un ZIP începe cu "PK\x03\x04":
i = date.find(b"PK\x03\x04")
if i != -1:
    open("ascuns.zip","wb").write(date[i:])
    print("arhiva extrasa de la octetul", i)
```

CE FACE

Mai multe fișiere pot stă lipite într-unul singur.

.find caută semnătura celui de-al doilea (aici PK = ZIP).

Tai de la semnătură încolo și salvezi: ai extras fișierul.

🎯 LA CE FOLOSEȘTE

Un clasic de steganografie: o poză care, de la un punct, conține de fapt o arhivă. Ochiul vede o poză; octeții spun altceva. Unealtă „binwalk" face exact căutarea asta, automat.



PARTEA V

Ascuns la vedere

Steganografie: LSB într-o imagine, cum o ascunzi, cum o dezvălui, cum se detectează.

Nu codat, nu criptat: pitit.

Un mesaj poate stă chiar în pixeli.

IDEEA · LSB

Bitul care nu se vede



LSB =

bitul din coadă.

```
>_ idee

# un canal de culoare = un octet, 0..255
rosu = 0b1100101 0   <- ultimul bit
#            ^ schimbat: 0 sau 1
# ochiul NU vede diferenta 100 vs 101
# dar acolo putem ascunde un bit de mesaj
```

Fiecare pixel are roșu, verde, albastru, fiecare un octet. Ultimul bit schimbă culoarea cu una dintr-o sută: invizibil pentru ochi, dar un loc perfect de ascuns informație.

👻 **PYTHON · ASCUNZI**

Pui un mesaj în ultimii biți

```
>_ python

from PIL import Image
img = Image.open("gazda.png").convert("RGB")
px = list(img.getdata())   # [(r,g,b), ...]
biti = "".join(f"{b:08b}" for b in
              b"secret")   # mesaj -> biti
# pui câte un bit în ultimul bit al fiecarui
# canal:  r = (r & ~1) | bit
```

CE FACE

Transformi mesajul în octeți, apoi în biți.

Parcurgi canalele pixelilor și scrii câte un bit în ultimul bit.

r & ~1 pune ultimul bit pe 0; | bit scrie bitul tău.

🎯 LA CE FOLOSEȘTE

Rezultatul arată identic cu poză originală, dar poartă mesajul. Ca să-l poată citi cineva, trebuie să știe regulă (câți biți, unde se termină): fără ea, e invizibil.

🔎 PYTHON · DEZVĂLUI

Citești biții înapoi

```
>_ python

px = list(Image.open("stego.png")
          .convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    biti += str(r & 1)   # ultimul bit
# regrupezi câte 8 biti -> octeți -> text
```

CE FACE

r & 1 extrage doar ultimul bit al canalului.

Aduni biții în ordine, apoi îi regrupezi câte 8.

Fiecare grup de 8 biți e un octet al mesajului ascuns.

🎯 **LA CE FOLOSEȘTE**

Dezvăluirea e inversul ascunderii: aceiași pași, citiți invers. Când apare text citibil, ai găsit mesajul. Când nu, fie nu e stego LSB, fie regulă e altă.

👁 PYTHON · DETECȚIE

Cum vezi că o poză „ascunde" ceva

```
>_ python

# 1. vizual: diferente, zgomot în zone plate
# 2. dimensiune: poză prea mare față de continut
# 3. statistic: distributia ultimilor biti
from collections import Counter
ultimii = [r & 1 for (r,g,b) in px]
print(Counter(ultimii))   # 50/50 = suspect
```

CE FACE

Într-o poză normală, ultimii biți nu sunt perfect 50/50.

Un mesaj ascuns îi echilibrează artificial: un semnal statistic.

Counter îți arată distribuția, exact ca la analiză de trafic.

🎯 LA CE FOLOSEȘTE

Nu „vezi" mesajul, dar vezi urma lui: anomalia. Așa lucrează detecția, în stego și în securitate în general: cauți ce iese din tipar, nu conținutul în sine.

UNELTE REALE · FORENSICS ȘI STEGO

Trusă pe care o vei vedea peste tot

Pe challenge-urile de forensics, reflexul e: file, strings, binwalk, exiftool. Apoi stego, dacă e poză.



binwalk

găsește și extrage fișiere ascunse în fișiere



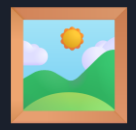
exiftool

citește și șterge metadate, orice format



zsteg / steghide

ascunde și scoate date din imagini



stegsolve

desface imaginea pe planuri de biți, vizual



strings / file

text citibil și tip real al fișierului



Pillow / binascii

aceleași operații, în Python

✋ ÎNTREBARE PENTRU SALĂ

Forensics pe telefonul sau laptopul altcuiva, fără acordul lui, e „doar curiozitate"?

Nu. E acces neautorizat la date, o infracțiune. Aceleași unelte, pe propriile fișiere sau cu mandat: muncă legală.



PARTEA V·

Produsul

Strângi encoding, XOR și identificarea într-un modul, cu un test.

Cinci tehnici, o singură unealtă.

Decoderul universal intră în arsenal.

🧰 PYTHON · ARSENAL

Un decoder care încearcă singur

>_ python

```python
# arsenal/forensics/decode.py
import base64
def incearca(s: str):
    try:
        b = base64.b64decode(s, validate=True)
        if all(32 <= c < 127 for c in b):
            return "base64", b.decode()
    except Exception: pass
    # ... la fel pentru hex, apoi XOR brute
    return "necunoscut", s
```

CE FACE

O funcție care încearcă stratul cel mai probabil și verifică rezultatul.

validate=True refuză base64 fals; filtrul „e citibil" confirmă reușita.

Întoarce ce strat a fost și ce a ieșit: intră direct în raport.

🎯 LA CE FOLOSEȘTE

Asta e regulă produsului: fiecare tehnică de azi devine o funcție reutilizabilă, nu un script de unică folosință. Dată viitoare, decoderul tău desface singur ce prinzi.

🧪 PYTHON · TESTUL

O unealtă în care ai încredere are un test

```
>_ python

# teste/test_decode.py
from arsenal.forensics.decode import incearca

def test_base64():
    tip, txt = incearca("U2FsdXQ=")
    assert tip == "base64"
    assert txt == "Salut"
```

CE FACE

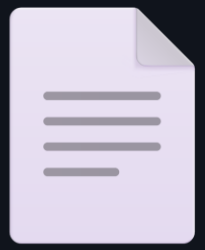
Testul fixează comportamentul: dat un input cunoscut, un output cunoscut.

Dacă mâine strici decoderul, testul pică imediat.

Rulezi cu pytest; „passed" e dovada, nu „pare că merge".

🎯 LA CE FOLOSEȘTE

Un test scurt e diferența dintre o unealtă pe care te bazezi și una în care speri. La examen, arsenalul tău rulează cu teste verzi, nu cu „cred că funcționa".



Raportul crește cu fiecare strat.
La examen, e deja scris.

Sesiunea de azi adaugă o secțiune: ce șiruri ai decodat, ce hash-uri ai identificat, ce ai găsit ascuns într-un fișier sau o poză, și cum s-ar apăra cineva. Nu la final, pe parcurs.

BAGAJUL DE AZI

Azi, pe două coloane

PROGRAMARE (CUM SCRII)	SECURITATE (CUM MERGE)	UNDE
base64, hex, unquote, bytes	recunoști stratul, îl desfaci	straturi
operatorul ^, buclă pe octeți	XOR, spargere prin forță brută	cheia
hashlib, dicționar, hmac	hash, spargere, salt, integritate	amprente
open "rb", regex pe bytes, PIL	magic bytes, strings, EXIF/GPS	urme
biți, Counter, funcție + test	stego LSB, detecție, arsenal	ascuns

Aceeași muncă, două fețe: scrii cod pe octeți și înțelegi ce ascunde sau dezvăluie, de fapt.

✋ ÎNTREBARE PENTRU SALĂ

Cu ce pleci azi?

Reflexul de a citi octeți, nu extensii; de a întreba „ce strat e?" înainte să decodezi; și un decoder care, pus în arsenal, desface singur ce prinzi dată viitoare.



Ai învățat să citești ce nu se vede: straturi, chei, amprente, urme.

În laboratorul acestei săptămâni: un decoder universal cu identificare și spargere (Lab A) și forensics pe fișiere și o poză, până la mesajul ascuns (Lab B). Pe fișiere pregătite pentru laborator, cod și output.